Java Streams — Interview Pattern Playbook

The "Blind 75" of Java Streams. ~18 patterns that cover almost every stream question you'll be asked. Each one: the trigger phrase to recognize it, minimal working code, and the one-line reasoning to say out loud in an interview.

---

1. Group by key

Trigger: "group X by Y" / "bucket employees by department"

```
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept));
```

Say: "groupingBy gives O(n) bucketing — key is whatever defines the group."

2. Group by key → count

Trigger: "how many X per Y" / word frequency

```
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept, Collectors.counting()));
```

3. Group by key → downstream aggregate

Trigger: "average/sum/max salary per department"

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.averagingDouble(Employee::getSalary)));

Map<String, Optional<Employee>> topEarnerByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDept,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))));
```

Say: "groupingBy's 2-arg overload takes a downstream collector — this composes instead of grouping then looping again."

4. Order-preserving group (LinkedHashMap trick)

Trigger: "first repeating X" / "preserve insertion order while grouping"

```
Map<Character, Long> ordered = str.chars().mapToObj(c -> (char) c)

    .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
Collectors.counting()));
```

Say: "default groupingBy uses HashMap — no order guarantee. Swap the map supplier when order matters."

5. Map to a single value per key (toMap)

Trigger: "build a lookup / id → name map"

```
Map<Integer, String> idToName = employees.stream()

    .collect(Collectors.toMap(Employee::getId, Employee::getName));
```

Say: "toMap throws IllegalStateException on duplicate keys unless you pass a merge function as the 3rd arg."

```
Map<String, Double> deptTotalSalary = employees.stream()

    .collect(Collectors.toMap(Employee::getDept, Employee::getSalary, Double::sum));
```

6. Partition into exactly 2 groups

Trigger: "pass/fail" / "even/odd" / boolean split

```
Map<Boolean, List<Employee>> partitioned = employees.stream()

    .collect(Collectors.partitioningBy(e -> e.getSalary() > 60000));
```

Say: "partitioningBy always returns both true/false keys, even if one bucket is empty — groupingBy wouldn't."

7. Flatten nested collections

Trigger: "list of lists" / "all orders across all customers"

```
List<String> allSkills = employees.stream()
        .flatMap(e -> e.getSkills().stream())
        .distinct()
        .collect(Collectors.toList());
```

Say: "map gives Stream<Stream>; flatMap merges into Stream — this is the #1 stream question after groupingBy."

8. Top-N / Nth highest

Trigger: "second highest salary" / "top 3 scores"

```
List<Double> top3 = employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .limit(3)
        .collect(Collectors.toList());
```

Say: "distinct before sorted+limit, or duplicates skew the ranking."

9. Single reduction (sum/min/max/custom)

Trigger: "total X" / "find the employee with max salary"

```
double total = employees.stream().mapToDouble(Employee::getSalary).sum();

Optional<Employee> highestPaid = employees.stream()
        .max(Comparator.comparingDouble(Employee::getSalary));
```

Say: "prefer built-in min/max/sum over reduce() unless the combine logic is genuinely custom — reduce is the fallback, not the default."

```
// true custom reduce
```

```
int totalChars = words.stream().reduce(0, (acc, w) -> acc + w.length(), Integer::sum);
```

10. Sort by multiple fields

Trigger: "sort by dept, then by salary desc"

```
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(Employee::getDept)
        .thenComparing(Employee::getSalary, Comparator.reverseOrder()))
    .collect(Collectors.toList());
```

11. Distinct with custom equality (objects, not primitives)

Trigger: "unique employees by id" (when equals/hashCode isn't overridden)

```
Collection<Employee> uniqueById = employees.stream()
    .collect(Collectors.toMap(Employee::getId, e -> e, (a, b) -> a, LinkedHashMap::new))
    .values();
```

Say: "distinct() relies on equals/hashCode — if you can't touch the class, fake distinct-by-key with toMap's merge function."

12. Statefull scan across elements (recognize when NOT to use pure streams)

Trigger: "first repeating char" / "running total" / anything needing memory of prior elements

```
// correct instinct: either plain loop, or externalize state via a collector (see #4)
// AVOID: .filter(x -> !seenSet.add(x)) — stateful lambda, anti-pattern, breaks under parallel()
```

Say: "streams model stateless transformations — if the logic needs to remember previous elements, that's a signal for a loop or a collector like groupingBy, not a captured mutable variable in filter/map."

13. Joining strings

Trigger: "comma separated list of names"

```
String csv = employees.stream().map(Employee::getName).collect(Collectors.joining(", ", "[", "]"));
```

14. Filter + count / anyMatch / allMatch

Trigger: "how many employees earn > 60k" / "does any employee..."

```
long count = employees.stream().filter(e -> e.getSalary() > 60000).count();

boolean anyHighEarner = employees.stream().anyMatch(e -> e.getSalary() > 100000);

boolean allPaidAboveMin = employees.stream().allMatch(e -> e.getSalary() > 30000);
```

Say: "anyMatch/allMatch/noneMatch short-circuit — prefer over filter().count() > 0 for large streams."

15. Summary statistics

Trigger: "min, max, avg, count all at once"

```
DoubleSummaryStatistics stats = employees.stream()
        .mapToDouble(Employee::getSalary)
        .summaryStatistics();
// stats.getMin(), getMax(), getAverage(), getCount(), getSum()
```

Say: "avoids 4 separate passes over the stream for 4 separate metrics."

16. Primitive streams (IntStream/DoubleStream) vs boxed

Trigger: any numeric aggregation

```
int[] arr = {5, 3, 8, 1};

int sum = Arrays.stream(arr).sum();          // IntStream — no boxing

IntStream.rangeClosed(1, 5).forEach(System.out::println);
```

Say: "mapToInt/mapToDouble avoid autoboxing — always prefer over Streamfor numeric work; this is a common 'why is this slow' interview follow-up."

17. Collect into a specific structure

Trigger: "return a Set" / "return a TreeMap sorted by key"

```
Set<String> names = employees.stream().map(Employee::getName).collect(Collectors.toSet());

Map<String, Employee> sortedByName = employees.stream()

    .collect(Collectors.toMap(Employee::getName, e -> e, (a, b) -> a, TreeMap::new));
```

18. Parallel stream — when it's actually safe

Trigger: "make this faster" / "use parallelStream"

```
double total = employees.parallelStream().mapToDouble(Employee::getSalary).sum();
```

Say: "safe only for stateless, associative, non-order-dependent operations on large enough datasets — never with a shared mutable collector like ArrayList::add via forEach; use collect(), which is parallel-safe by design."

The interview answer template (say this structure out loud)

1. **Identify the shape** — is this group/reduce/partition/flatten/top-N?
2. **Pick the collector** — name it before writing code ("I'll use groupingBy with a counting downstream collector").
3. **Flag the trap** — duplicates? ordering? stateful logic? boxing?
4. **Write it, then state the complexity** — O(n) almost always, but call out extra passes (e.g., distinct + sorted = 2 passes conceptually).

Interviewers weight step 2–3 more than syntax recall — pattern-naming out loud is what reads as senior.

Practice rotation (do each pattern in both loop and stream form)

- Patterns 1–6: LeetCode "Group Anagrams", "Top K Frequent Elements"
- Pattern 7: any "flatten list of lists" / nested JSON-like structure problem
- Pattern 8: "Kth largest element"

- Pattern 12: "First non-repeating character", "Longest substring without repeating characters"
- Pattern 9–10: any employee/product dataset sort-and-aggregate problem (very common in Java-specific interviews, less so in generic LeetCode)